

CS 170 Midterm 1

Write in the following boxes clearly and then double check.

Name :

SID :

Exam Room :

☐ DWIN 155 ☐ VLSB 2050 ☐ EVAN 10
☐ Other (Specify):

Name of student to your left :

Name of student to your right :

- The exam will last 110 minutes.
- The exam has 11 questions with a total of 120 points. You may be eligible to receive partial credit for your proof even if your algorithm is only partially correct or inefficient.
- Only your writings inside the answer boxes will be graded. **Anything outside the boxes will not be graded.** The last page is provided to you as a blank scratch page.
- Answer all questions. Read them carefully first. Not all parts of a problem are weighted equally.
- Be precise and concise.
- The problems may **not** necessarily follow the order of increasing difficulty.
- The boxes assigned to each problem are by no means an indication of the problem's difficulty.
- Unless the problem states otherwise, you should assume constant time arithmetic on real numbers. Unless the problem states otherwise, you should assume that graphs are simple.
- If you use any algorithm from lecture and textbook as a blackbox, you can rely on the correctness and time/space complexity of the quoted algorithm. If you modify an algorithm from textbook or lecture, you must explain the modifications precisely and clearly, and if asked for a proof of correctness, give one from scratch or give a modified version of the textbook proof of correctness.
- Assume the subparts of each question are **independent** unless otherwise stated.
- Please write your SID on the top of each page. **Doing this will be worth 1 point on the exam.**
- Good luck!

1 Asymptotic Analysis (4 points)

For each pair of functions f and g , specify whether $f = O(g)$, $g = O(f)$, or both. Assume all logarithms are to base 2.

f	g	$f = O(g)$	$g = O(f)$	Both
$2^{\log n}$	$n^{0.9}$	☐	☐	☐
$n^4 + 2^n$	$n^2 + 3^n$	☐	☐	☐
$(\log n)^2$	$n^{1/2}$	☐	☐	☐
$2n^4 + n^2 \log n$	$n^4 + 10n^3$	☐	☐	☐

Solution:

1. $g = O(f)$ only. Note that $2^{\log n} = n$ since the logarithm is base 2.
2. $f = O(g)$ only. The exponential 2^n and 3^n terms dominate, and $2^n = O(3^n)$ but $3^n \neq O(2^n)$.
3. $f = O(g)$ only. Recall that polynomials always dominate logarithms.
4. Both $f = O(g)$ and $g = O(f)$. The $2n^4$ and n^4 dominate, and they are a constant factor off of one another.

2 Runtime Analysis (10 points)

Consider the following piece of code:

```

Function what( $n$ ) {
    If ( $n < 2$ ):
        return
     $k \leftarrow n$ 
    while  $k > 1$  do {
        for  $j = 1, 2, 3, \dots, k$  {
            print  $n$ 
        }
         $k \leftarrow \lfloor 3 * k / 4 \rfloor$ 
    }
    what( $\lfloor n/2 \rfloor$ )
    what( $\lfloor n/2 \rfloor$ )
}

```

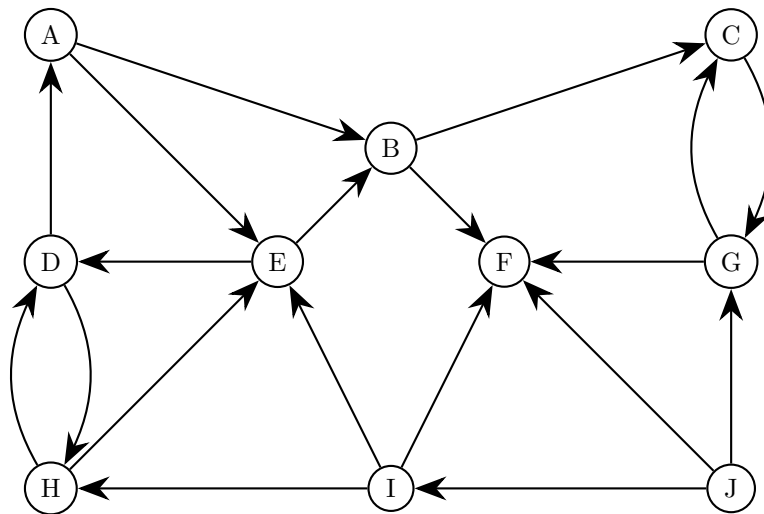
Here $\lfloor \cdot \rfloor$ denotes the floor function that rounds real numbers down to integers

Let $B(n)$ denote the number of times the number n is printed by a call to `what( $n$ )`. Let $T(n)$ denote the total runtime of the `what( $n$ )`.

1. $B(n) = \Theta\left(\boxed{}\right)$

2. Briefly justify the estimate of $B(n)$.

3 Connectivity in Graphs (8 points)



1. Perform DFS in the graph starting at node A and write down the pre and post numbers. Break ties alphabetically.

Vertex v	$pre[v]$	$post[v]$
A		
B		
C		
D		
E		
F		
G		
H		
I		
J		

Solution:

Vertex v	$pre[v]$	$post[v]$
A	1	16
B	2	9
C	3	8
D	11	14
E	10	15
F	5	6
G	4	7
H	12	13
I	17	18
J	19	20

2. Mark all cross and back edges if any, by filling the circles below.

Edge	Fill if cross or back edge
$A \rightarrow B$	☐ Cross ☐ Back
$A \rightarrow E$	☐ Cross ☐ Back
$B \rightarrow F$	☐ Cross ☐ Back
$B \rightarrow C$	☐ Cross ☐ Back
$C \rightarrow G$	☐ Cross ☐ Back
$D \rightarrow A$	☐ Cross ☐ Back
$D \rightarrow H$	☐ Cross ☐ Back
$E \rightarrow B$	☐ Cross ☐ Back
$E \rightarrow D$	☐ Cross ☐ Back

Edge	Fill if cross or back edge
$G \rightarrow C$	☐ Cross ☐ Back
$G \rightarrow F$	☐ Cross ☐ Back
$H \rightarrow D$	☐ Cross ☐ Back
$H \rightarrow E$	☐ Cross ☐ Back
$I \rightarrow E$	☐ Cross ☐ Back
$I \rightarrow F$	☐ Cross ☐ Back
$I \rightarrow H$	☐ Cross ☐ Back
$J \rightarrow G$	☐ Cross ☐ Back
$J \rightarrow I$	☐ Cross ☐ Back
$J \rightarrow F$	☐ Cross ☐ Back

Solution: Cross edges: $\{E \rightarrow B, I \rightarrow E, I \rightarrow F, I \rightarrow H, J \rightarrow G, J \rightarrow I, J \rightarrow F\}$

Back edges: $\{D \rightarrow A, G \rightarrow C, H \rightarrow D, H \rightarrow E\}$

3. In this class, we have learnt an algorithm to compute the strongly connected components (SCCs) of a directed graph. Write down the strongly connected components (SCCs) in the above graph in the same order in which the algorithm outputs them.

Note that there may be more answer boxes below than SCCs in the graph.

SCC 1:		SCC 2:	
SCC 3:		SCC 4:	
SCC 5:		SCC 6:	
SCC 7:		SCC 8:	

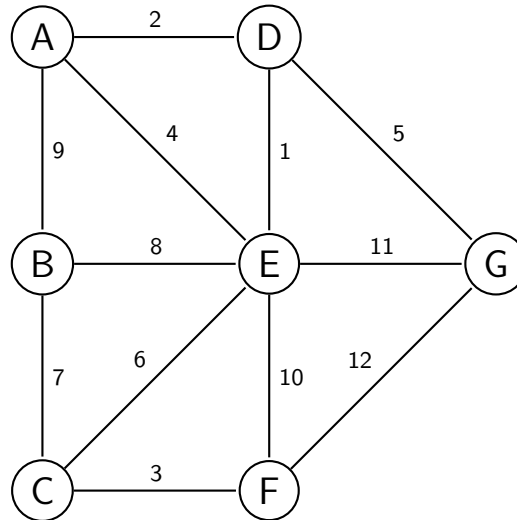
Solution: The SCCs are: $\{A, D, E, H\}, \{B\}, \{I\}, \{J\}, \{C, G\}, \{F\}$. Kosarajus algorithm outputs them in reverse topological order, so the right order is: $\{F\}, \{C, G\}, \{B\}, \{A, D, E, H\}, \{I\}, \{J\}$.

Solution:

deleteMin Steps	<i>(value, key)</i> -pairs in priority queue				
0	$(S, 0)$	(A, ∞)	(B, ∞)	(C, ∞)	(D, ∞)
1	$(S, \boxed{})$	$(A, 15)$	$(B, 5)$	$(C, 12)$	(D, ∞)
2	$(S, \boxed{})$	$(A, 15)$	$(B, \boxed{})$	$(C, 10)$	(D, ∞)
3	$(S, \boxed{})$	$(A, 11)$	$(B, \boxed{})$	$(C, \boxed{})$	$(D, 20)$
4	$(S, \boxed{})$	$(A, \boxed{})$	$(B, \boxed{})$	$(C, \boxed{})$	$(D, 20)$

5 MST Execution (6 points)

1. In the graph below, what would be the fourth and fifth edges added by Kruskal's algorithm?

Fourth edge: Fifth edge:

Solution:

Fourth edge: *DG* Fifth edge: *CE*

2. In the same graph, what would be the fourth and fifth edges added by Prim's algorithm started at *A*?

Fourth edge: Fifth edge:

Solution:

Fourth edge: *CE* Fifth edge: *CF*

3. In the same graph, Dijkstra's algorithm can be used to construct a shortest path tree starting from *A*. What would be the fourth and fifth edges added by the Dijkstra's algorithm to the shortest path tree:

Fourth edge: Fifth edge:

Solution:

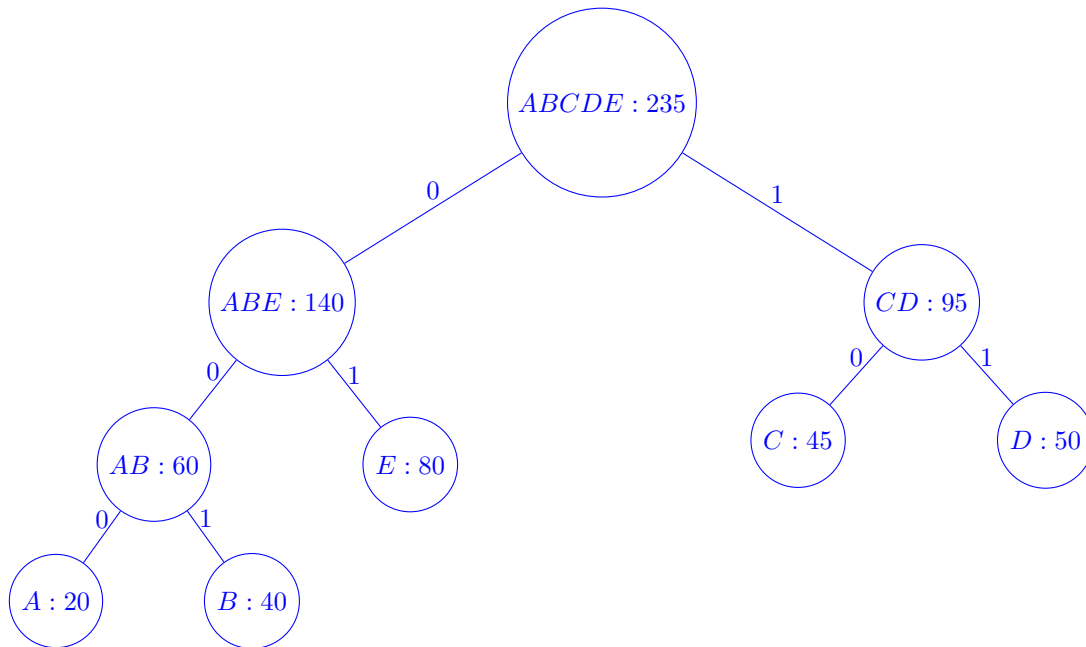
Fourth edge: *AB* Fifth edge: *CE*

The two edges would be tied, as before the 4th node is popped from the queue, both nodes *B* and *C* would have distance 9 from *A*. Therefore, either order is acceptable.

6 Short Answers (18 points)

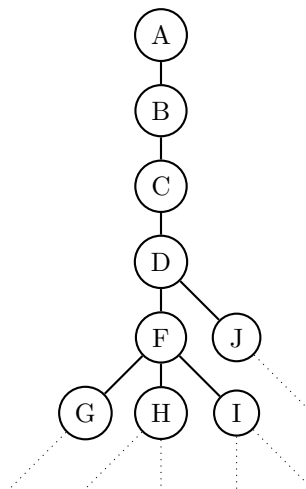
1. (2 points) A document consists of symbols A, B, C, D, E with frequencies 20, 40, 45, 50, 80 respectively. Suppose we run Huffman encoding on these letters, the letter C will be encoded using bits, and the letter E is encoded using bits.

Solution: After running the Huffman encoding algorithm, we obtain the following Huffman tree:



Hence, we see that C is encoded using 2 bits, and E is encoded using 2 bits.

2. (3 points) Here is DFS tree for an undirected connected graph, shown up to depth 5. The vertex A is the root of the tree.



(Warning: The picture shows the tree only up to depth 5, and does not show any of the non-tree edges in the graph. The dotted lines at the bottom indicate subtrees hanging below).

Which is the smallest set of vertices whose deletion will necessarily disconnect the graph?

Briefly justify why deletion of this set disconnects the graph:

Solution: The smallest set of vertices whose deletion *must* delete the graph is $\{A, B, C, D\}$.

We know that the subtree rooted at J has no edges going to node F or any of the subtrees rooted at G , H , or I .

However, it's possible that J , F , G , H , I , or their corresponding rooted subtrees could have edges to nodes A , B , C , or D .

Deleting only node D may not necessarily disconnect the graph, as the subtree rooted at J could be connected to the subtree rooted at F via some path going through nodes A , B , or C .

3. (2 points) In a weighted undirected graph, G , if edges e_1 and e_2 have the same weight, and e_1 is in every minimum spanning tree of G , then e_2 must be in at least one minimum spanning tree of G . (Assume that the graph is connected and has at least 3 nodes)

☐ True ☐ False

Solution: False. We could have a scenario where e_1 is a critical edge and e_2 is the heaviest edge in a cycle.

4. (2 points) In a weighted undirected graph G with distinct edge weights, the second lightest edge is always part of an MST. (Assume that the graph is connected and has at least 3 nodes)

☐ True ☐ False

Solution: True. Let e_1 be the lightest edge and e_2 be the second lightest edge. There is a cut that e_2 crosses but that e_1 doesn't cross (since e_1 and e_2 differ in at least one vertex). By the cut property, e_2 must be part of an MST.

5. (2 points) Let G be a connected undirected graph with n vertices and m edges. In every DFS traversal of the graph, the number of back edges is , while the number of cross edges

is .

Solution: Note that in an undirected graph, all non-tree edges are back edges. Hence, the number of back edges is $m - (n - 1) = m - n + 1$, and the number of cross edges is 0.

6. (1 point) The fastest known algorithm to add two $n \times n$ matrices takes $\Theta\left(\text{}\right)$ time. Here assume additions and multiplications of individual numbers take constant time.

Solution: The fastest known algorithm to add two $n \times n$ matrices takes $\Theta(n^2)$ time. We must handle all $2n^2$ entries with n^2 individual operations (since they are independent of each other).

7. (2 point) Let e be an edge in a connected undirected graph G . The graph G has n vertices and m edges. Deleting the edge e can break the graph into a maximum of connected components.

Solution: 2. First, clearly 2 is possible. To show that it's maximal, consider a spanning tree T of the graph. If e is in the spanning tree, $G \setminus \{e\}$ will still include T but will be connected. Otherwise, if $e \in T$, then removing e from T will disconnect the tree into at most 2 components, and therefore the graph into at most 2 components.

8. (2 point) Let e be an edge in a strongly connected directed graph G . The graph G has n vertices

and m edges. Deleting the edge e can break the graph into a maximum of strongly connected components.

Solution: n . If the starting graph is a directed cycle, then deleting any edge would break the cycle and result in every vertex becoming its own SCC.

9. (1 point) In every directed graph G , any vertex v can have the largest pre-value ($pre[v]$) in a DFS traversal, depending on the order of vertices chosen by the traversal.

☐ True ☐ False

Solution: False. Consider the graph G with vertices $V = \{A, B, C, D\}$ and edges $E = \{(A, B), (B, C), (C, D), (D, B)\}$; then, vertex B can never have the largest pre-value.

10. (1 point) In every directed graph G , any vertex v can have the smallest pre-value ($pre[v]$) in a DFS traversal, depending on the order of vertices chosen by the traversal.

☐ True ☐ False

Solution: True. We can always choose to start the DFS at v , which would give it the smallest pre-value.

7 Fibonacci Revisited (12 points)

Consider the following simple algorithm to compute the n^{th} -Fibonacci number. Here $F[i]$ stores the i^{th} -Fibonacci number.

Simple-Fibonacci(n)

```

     $F[1] \leftarrow 1$ 
     $F[2] \leftarrow 1$ 
    for  $i = 3$  to  $n$ 
         $F[i] \leftarrow F[i-1] + F[i-2]$ 
    return  $F[n]$ 

```

Recall that the n^{th} Fibonacci number F_n is $\Theta(n)$ -digits long. Therefore, a correct implementation of the above algorithm will actually need an array of digits to store each $F[i]$. In other words, for each i , $F[i]$ is itself an array of digits of length up to n .

Throughout this question, we assume that operations on individual digits take $\Theta(1)$ time. But arithmetic operations on n digit numbers are implemented using operations on digits, and may take time depending on n .

1. The total runtime of the above algorithm is $\Theta\left(\boxed{}\right)$.
2. Briefly justify your answer.

Solution:

1. $\Theta(n^2)$
2. We have the recurrence relation $T(n) = T(n-1) + \Theta(n)$. Using the unraveling method gives $T(n) = T(2) + 3 + 4 + \dots + n \in \Theta(n^2)$.

Now, we will devise a faster algorithm to compute the n^{th} -Fibonacci number. Towards this, we will first derive a different formula for the n^{th} Fibonacci number.

Let $A[i]$ denote the following vector with two coordinates:

$$A[i] = \begin{bmatrix} F[i+1] \\ F[i] \end{bmatrix}$$

and let M denote the following 2×2 matrix:

$$M = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$$

Then we have the following identity:

$$A[i] = MA[i-1]$$

Here $MA[i-1]$ denotes the multiplication of the 2×2 -matrix M , with the 2×1 vector $A[i-1]$. By induction, we have the following formula for the Fibonacci numbers,

$$A[n] = M^{n-1} \cdot \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

In the above formula, M^{n-1} is the matrix multiplied $n-1$ times with itself, i.e., $M^{n-1} = \underbrace{M \cdot M \cdot \dots \cdot M}_{n-1 \text{ times}}$

3. Using the above formula for the Fibonacci numbers, describe how to compute the n^{th} Fibonacci number asymptotically faster than the **Simple-Fibonacci** algorithm. (As always, you may use any algorithm we covered in textbook, lecture or discussions as a black-box).

Solution: Note that if we can calculate M^{n-1} , then $M^{n-1}A[0]$ gives $F[n]$ in the top entry. In order to compute this efficiently, we use a divide-and-conquer approach to compute a given M^k :

- If k is odd, then we first calculate $M^{(k-1)/2}$, square this matrix, then multiply by M .
- If k is even, we first calculate $M^{k/2}$ then square it.

The largest entry in M^k will have $\Theta(k)$ digits. Squaring M^k requires 8 scalar multiplications, and if we call Karatsuba's multiplication algorithm, this takes $\Theta(k^{\log_2 3})$ time. So, the recurrence relation is:

$$T(n) = T(n/2) + \Theta(n^{\log_2 3}).$$

By Master Theorem, this algorithm takes $\Theta(n^{\log_2 3})$ operations to calculate M^{n-1} . Finally, once M^{n-1} is computed, the product $M^{n-1}A[0]$ can be computed using 4 scalar multiplications of $\Theta(n)$ -bit numbers, so also $\Theta(n^{\log_2 3})$ using Karatsuba's.

8 Four Cycles (10 points)

Let A denote the adjacency matrix of an undirected graph G on n vertices. Let $B = A^2 = A \cdot A$ where $A \cdot A$ denotes multiplication of matrices.

1. The ij^{th} entry of matrix B in terms of entries of matrix A is:

$B[i, j] =$

Solution:

$$B[i, j] = \sum_{k=1}^n A[i, k] \cdot A[k, j]$$

where $\cdot$ represents scalar multiplication.

2. If $B[i, j] > 0$ for some $i \neq j$, then what can you say about vertex i and vertex j ? (*Hint: Carefully observe the expression for $B[i, j]$ from the first part*)

Solution: If $B[i, j] > 0$, we know there exists some path $i \rightarrow k \rightarrow j$ where $k \neq \{i, j\}$ (that is, there is a simple path of length 2 from i to j).

3. A 4-cycle in a graph G consists of 4 vertices u_1, u_2, u_3, u_4 such that $(u_1, u_2), (u_2, u_3), (u_3, u_4)$ and (u_4, u_1) are all edges. Describe an algorithm to check if a graph has a 4-cycle in time strictly faster than $O(n^{2.99})$. Your algorithm only needs to detect the existence of the 4-cycle—it doesn't need to find one.

Solution: We calculate A^2 and then check to see if there is an off-diagonal entry that is at least 2. If the (i, j) th entry from A^2 is at least 2, then there are at least two distinct vertices, call them k_1, k_2 , such that $A[i, k_1] \cdot A[k_1, j] = 1$ and $A[i, k_2] \cdot A[k_2, j] = 1$. Since we have that $A[i, i] = 0$, this means that $i \rightarrow k_1 \rightarrow j$ and $i \rightarrow k_2 \rightarrow j$ are both paths. Since A is undirected, $i \rightarrow k_1 \rightarrow j \rightarrow k_2 \rightarrow i$ is a four-cycle.

Otherwise, if all off-diagonal entries from A^2 are at most 1, then for every i and j , there is at most one k such that (i, k) and (k, j) are edges, meaning that there is no four cycle with i and j as opposite vertices.

Using Strassen's algorithm, we can calculate A^2 in $O(n^{\log_2 7})$ time, which is faster than $O(n^{2.99})$.

9 Bellman-Ford Algorithm (5 points)

Recall that Bellman-Ford algorithm for shortest paths in a graph has the following form:

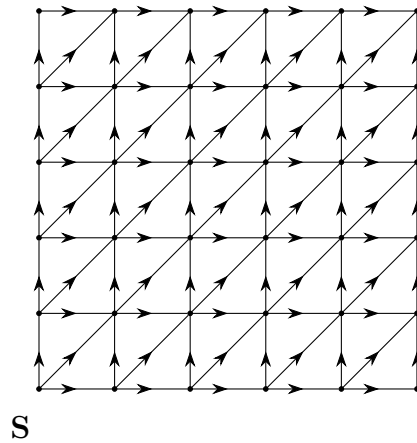
```

(*Outer Loop*)    for  $t = 1$  to  $|V| - 1$ 
  (*Inner loop*)  for each edge  $e \in E$ 
                  update(edge  $e$ )

```

Here “update(edge e)” modifies the distance values to the nodes in the graph.

Consider the execution of Bellman-Ford algorithm on the following **directed** graph with **positive or negative** edge weights and the source node S . The graph is a grid, with S being the bottom left corner and all edges directed up or to the right, or diagonally.



The convergence of the distance values during the execution of the algorithm depends on the order in which the edges are updated in the inner loop.

- For the above graph, with vertex S as starting vertex, what is the minimum number of iterations of the outer loop, after which the distance values to all vertices are guaranteed to be correct, regardless of the edge weights, for the **worst possible ordering** of the edge updates?

Solution: 10. Note that after the k -th Bellman-Ford iteration, we guarantee correctness of all distances values that are at least k edges away from the source vertex. Thus, in the worst case, we need to go through at least L iterations, where L is the longest path from the source to any vertex in the graph.

- For the above graph, with vertex S as starting vertex, what is the minimum number of iterations of the outer loop, after which the distance values to all vertices are guaranteed to be correct, regardless of the edge weights, for the **best possible ordering** of the edge updates?

Solution: 1. Since the graph is a DAG, we can update the edges in topological order (based on vertices) to guarantee correctness for all distance values in just a single iteration.

10 Electric Cars (20 points)

Given the road network of the state of Electrica, we want to compute the quickest route to go from a city S to a city T in an electric car. Unfortunately, the electric car needs to recharge every so often and charging takes time. So we will devise an algorithm to compute the quickest routes taking into account the charging stops along the way. Here are the details:

Formal description:

Input:

- An undirected graph $G = (\{1, \dots, n\}, E)$ representing the road network between cities in Electrica.
- For each edge $e = (i, j)$, a positive integer d_e representing the drive time in minutes, to go from city i to j along the edge.
- The car can be charged at a subset $S \subset \{1, \dots, n\}$ of cities. There are two options for charging that are summarized in the table below.

Name	Charging Time	Drive time
Full Charge	C_{full}	D_{full}
Partial Charge	$C_{partial}$	$D_{partial}$

So a *full charge* takes C_{full} minutes to complete, but allows the electric car to drive another D_{full} minutes. A *partial charge* takes $C_{partial}$ minutes to complete, but can allow the car to drive another $D_{partial}$ minutes.

The car can undergo a Full or Partial Charge while at a city in S , irrespective of its pre-existing level of charge. Furthermore, the charging times for the full or partial charge do not depend on the amount of pre-existing charge in the car.

- A source city s and a destination city t in the graph G

Output: The quickest route from the source s to destination t in the electric car. Here the *quickest route* is one that takes the shortest total time including both drive and charging times. Assume that the car starts with a *Full Charge* at the source city s .

1. Give a brief but precise description of an algorithm for the problem. (Your algorithm should run in a time that is strictly shorter than $\Theta(n^4)$. Partial credit if the runtime depends on the numerical parameters like $C_{full}, D_{full}, C_{partial}, D_{partial}$ and edge weights d_e)

(Answer Box on next page)

Solution:

Intuition: Consider the quickest path P from s to a city v . We can divide the path P into sections based on charging events.

So we can split the path P into sections

$$P = P_1 \cdot P_2 \cdot P_3 \cdots P_t$$

The first section P_1 starts at s and ends at some city u_1 with a charging station. The second section consists of charging either Partial/Full at city u_1 , and then taking a path P_2 to some city u_2 with a charging station. More generally, the i^{th} section P_i consists of charging Partial/Full at city u_{i-1} and taking a path to city u_i . The final section starts with a partial/full charge but ends at the destination v .

We will construct a "meta-graph" where each section is an "meta-edge" and compute the shortest path in the meta graph.

The nodes of the "meta-graph" are just all cities. Note that each section is just a shortest path with no charging in between. So we can compute the meta edges by Dijkstra executions on the original graph.

Here are the details. Run Dijkstra's algorithm from every charging city, as well as s , to find shortest path distances from each source. Denote $d_w(u, v)$ to be the shortest path distance from u to v in the SSSP tree with source w .

We then define the "charging distance" D from charging city u to an arbitrary city v to be

$$D(u, v) := \begin{cases} C_{\text{partial}} + d_u(u, v) & \text{if } d_u(u, v) \leq D_{\text{partial}} \\ C_{\text{full}} + d_u(u, v) & \text{if } D_{\text{partial}} < d_u(u, v) \leq D_{\text{full}} \\ \infty & \text{if } D_{\text{full}} < d_u(u, v) \end{cases}$$

(Here $D(u, v)$ is the length of the meta-edge from charging city u to city v)

In the case of starting from s , we set the charging distance to be

$$D(s, v) := \begin{cases} d_s(s, v) & \text{if } d_s(s, v) \leq D_{\text{full}} \\ \infty & \text{otherwise} \end{cases}$$

This defines a new directed graph $G' = (V', E', w : (u, v) \rightarrow \mathbb{R})$ with

$$\begin{aligned} V' &= \{s, t\} \cup S \\ E' &= \{(u, v) \in E : u, v \in V'\} \\ w(u, v) &= D(u, v). \end{aligned}$$

Running Dijkstra's on G' starting from s gives us the shortest path from s to t that includes both drive and charging times.

Alternative solution:

Run Dijkstras from s , but keep a 2D array of "distances and charges," that is for a given node we have an array of all paths to this node as well as the charge we have left at the end of each path. So the first path in this list will be the shortest path, the second one will be the shortest path with remaining charge greater than the other path, etc. (We add self loops in the charging cities, that take the respective time and assign the respective charge to the path as well). If a path would run out of charge, then the distance is ∞ and we can prune it.

When we process a candidate path at node u , if this path has more charge left than the last added path at node u , we add this path to the array, and relax the node a la Dijkstra.

This is harder to analyze (and we can't use a fibonacci heap!). Note that the amount of times we can find a "better charge" for a node is limited by the number of charging stations, aka $O(V)$. So each node is relaxed $O(V)$ times, resulting in an $O(|V||E| \log |V|)$ algorithm.

2. What is the run-time of your algorithm?

Solution: We provide a runtime analysis for the first solution. The first stage of Dijkstra calls takes $O(|V|) \cdot O(|E| + |V| \log |V|)$ time. The second stage takes $O(|V|^2)$ time. Hence, the overall runtime is

$$O(|V|(|E| + |V| \log |V|)),$$

which is $o(|V|^4)$.

Alice and Bob figure out the k^{th} -smallest element, one bit at a time. To find the most significant bit (highest bit), Alice sends Bob how many of her integers have 0 as its most-significant bit. Similarly, Bob announces to Alice how many of his integers have most significant bit 0. Let us say these numbers are n_A and n_B respectively.

If $k \leq n_A + n_B$, then they know that k^{th} smallest has highest bit 0. In this case, Alice and Bob throw away all integers with first bit 1, and repeat the same procedure for the smaller lists for the next bit, and so on.

If $k > n_A + n_B$ then the highest bit of k^{th} smallest element is 1. Alice and Bob throw away all integers with highest bit 0. In the shorter lists, they now look for $k - n_A - n_B$ th smallest element.

In 32 iterations, they learn all the bits. In each round, the integers n_A, n_B communicated are of length < 30 bits each. So total communication is at most $2 \cdot 30 \cdot 32 = 1920 < 4000$ bits.

Alternative Solution: Since we do not have to worry about computation, let's assume that both Alice and Bob sort their list of integers in ascending order. Doing some quick math, we get that Alice and Bob together can send up to $4000/32 = 125$ 32-bit integers for full credit.

Let's denote Alice's list by A , Bob's list by B , and the concatenation of the two lists as C . We also let $n = 2^{30}$.

The key idea is: *the median of C must be between the medians of A and B .*

Both the lists we are dealing with are even, so the median is not necessarily a number in the list and we need to handle the 2 median representatives. Suppose Alice sends $(A[n/2 - 1], A[n/2])$ to Bob, and Bob sends $(B[n/2 - 1], B[n/2])$. Then, they calculate $l := \max(A[n/2], B[n/2])$ and $r := \min(A[n/2 - 1], B[n/2 - 1])$ and figure out for both of them who had the larger (resp. smaller) "median". We break ties so that, when the two are tied, Alice's numbers are counted as larger.

Now there are two cases: either the same person has l and r , or one person has l and the other has r .

If the same person has l and r , we are done! WLOG suppose Alice has both l and r . Then we output $\frac{B[n/2-1] + B[n/2]}{2}$. The number of elements larger (resp. smaller) than this quantity is n both in B and in A , so we have found the median.

Now suppose we are in the other case, where Alice has r and Bob has l (the other way is symmetric). If Alice tosses out $r = A[n/2], A[n/2 + 1], \dots, A[n - 1]$ and Bob tosses out $B[0], \dots, B[n/2 - 1] = l$, then we have tossed out an equal number of elements on both sides of the median of C , so the median of the new (combined) list C' is the same as the old one, and now Alice and Bob have halved their list sizes! We then recurse.

Note that, if Alice and Bob have lists of size 2, then once they communicate they both know C , and hence can compute the median.

To see that this fits the communication requirement, note that each round communicates 4 integers, and we have at most $30 \cdot 4 = 120$ rounds, which is within the limit of 125.

You may use this page for scratch work; note that anything written on this page will be ungraded.